# Contención de incendios forestales con DQN y Double DQN sobre SimFire

Orellana César Andrés

Aprendizaje por Refuerzo I — Especialización en Inteligencia Artificial (UBA) — Desafío Nº 1

Repositorio: `https://github.com/OrellanaCesar/Trabajo-Final-Aprendizaje-Refuerzo-I`

Notebook: `incendios_dqn_TP_FINAL.ipynb` (ejecutable en Google Colab)

**Resumen**

En este trabajo entrené agentes DQN y Double DQN para contener un incendio forestal simulado con SimFire, colocando un cortafuego por turno sobre una grilla de $16 \times 16$. Cuento el problema, cómo lo resolví y, sobre todo, el camino que hice: los problemas de instalación del simulador, la calibración de sus parámetros, y la serie de pruebas y correcciones hasta que el agente aprendió. El resultado principal es que, con *reward shaping*, DQN y Double DQN bajan el área quemada de ∼142 celdas (política al azar) a ∼75, quedando cerca de una heurística simple (66) que no logran superar. No encontré una diferencia demostrable entre DQN y Double DQN, y un cuarto experimento (bonus decreciente) dio un resultado negativo que también explico.

## 1. Introducción y consigna

La consigna del Desafío Nº 1 pide resolver un problema real con alguna de las técnicas vistas en la materia, con estos requisitos: gráficos de convergencia (recompensa por episodio, mínimo 2), una comparación de al menos dos agentes, un informe en PDF de mínimo 3 carillas con lo más relevante del código, explicar el problema y los inconvenientes encontrados, y un script que funcione, de preferencia en Google Colab.

## 2. El problema y el entorno

### 2.1. SimFire

SimFire es el simulador de incendios del proyecto Fireline de MITRE. Modela la propagación con las ecuaciones de Rothermel, así que la velocidad del fuego depende del combustible, la humedad, el viento y la pendiente. Permite colocar tres tipos de mitigación (línea de fuego, de rascado y mojada); yo usé solo la línea de fuego.

Existe también SimHarness, un entorno de entrenamiento construido sobre SimFire, pero depende de Ray/RLlib y de Python 3.9, así que lo descarté y armé mi propio entorno con interfaz tipo Gym (`reset` y `step`) directamente sobre la clase `FireSimulation`.

### 2.2. Definición del problema

- **Mapa:** grilla de $16 \times 16$ celdas (256 acciones posibles), humedad 0.08, viento de 8 mph, `pixel_scale` 300 (valores que calibré, ver sección 4).

- **Escenarios:** en cada episodio cambian el punto de ignición (zona central del mapa) y la dirección del viento (8 direcciones). El combustible y la elevación son fijos.
- **Estado:** imagen de 9 canales de $16 \times 16$: 4 canales con el estado de cada celda (sin quemar, ardiendo, quemada, cortafuego), 1 de combustible, 1 de elevación y 3 de viento (velocidad, seno y coseno de la dirección).
- **Acción:** índice de la celda donde coloco el cortafuego. Después de cada acción el fuego avanza un minuto simulado. Enmascaro las celdas que ya están quemadas o tienen cortafuego, para que el agente no pierda tiempo con acciones sin sentido.
- **Recompensa:** $-0{,}1$ por cada celda que se prende fuego en el turno. Maximizar la recompensa equivale a minimizar el área quemada.
- **Fin del episodio:** cuando el fuego se apaga o se llega a 100 turnos. Si se acaban los turnos y el fuego sigue, lo dejo terminar sin más cortafuegos, para poder medir el área final.

## 3. Puesta en marcha de SimFire

Lo primero que hice fue ver si SimFire se podía instalar en Python 3.12, que es el que usa Colab. Encontré dos problemas:

1. **El paquete pide Python 3.9.** En PyPI todas las versiones de `simfire` declaran `python ~=3.9`, por lo que `pip install simfire` falla. Lo resolví instalándolo con `-nodeps --ignorerequirespython` y agregando a mano las dependencias que sí hacen falta (`noise`, `reportlab`, `svglib`, `wurlitzer`, `jsonlines`, `pygame`, entre otras).
2. **Hay módulos que no se pueden instalar.** SimFire importa `landfire`, `geotiff`, `geopandas` y `geopy`, que solo se usan para bajar mapas reales de Estados Unidos. `landfire` no tiene versión para Python 3.12, y los demás arrastraban dependencias que chocaban entre sí. Como yo uso terreno generado por funciones, los reemplacé por módulos vacíos antes de importar SimFire.

Con esos arreglos probé en un entorno limpio de Python 3.12 (con NumPy 2) y la simulación corrió bien. Esto es lo que hace el notebook al comienzo:

```
def modulo_falso(nombre, **atributos):
    modulo = types.ModuleType(nombre)
    modulo.__dict__.update(atributos)
    sys.modules[nombre] = modulo


modulo_falso("landfire")
modulo_falso("landfire.product")
modulo_falso("landfire.product.enums", ProductRegion=None, ProductTheme=None, ProductVersion=None)
modulo_falso("landfire.product.search", ProductSearch=None)
modulo_falso("geopandas")
modulo_falso("geopy")
modulo_falso("geopy.distance", great_circle=None)
modulo_falso("geotiff", GeoTiff=None)
```

## 4. Calibración del simulador

Antes de entrenar quise saber si el problema tenía sentido: que los cortafuegos realmente frenen el fuego, y que el fuego tarde lo suficiente como para que el agente pueda actuar. Hice pruebas cortas con el simulador, sin ningún agente, y de ahí salieron los valores del entorno. Estas pruebas las hice en una máquina local con scripts descartables, por eso sus números no están en el repositorio.

### 4.1. ¿Frenan los cortafuegos?

En una grilla de $24 \times 24$ probé poner una columna completa de cortafuegos delante del fuego. Con humedad 0.03 y viento de 15 mph el fuego quemaba los cortafuegos y el resultado era prácticamente igual a no poner nada (569 celdas quemadas contra 572). Con humedad 0.08 y viento de 8 mph la línea completa sí lo frenaba:

**Cuadro 1:** Efecto de una línea de cortafuegos con humedad 0.08 (grilla de $24 \times 24$).

| Situación | Celdas quemadas |
|---|---|
| Sin cortafuegos | 574 |
| Línea completa | 335 |
| Línea con un hueco | 549 |
| Línea diagonal | 549 |

Esto me dejó dos conclusiones: hay que usar humedad 0.08, y para contener el fuego no alcanza con poner cortafuegos sueltos, la línea tiene que quedar *cerrada* (un solo hueco, o una diagonal, deja pasar el fuego). Eso hace el problema más difícil para el agente.

### 4.2. ¿Qué tan rápido avanza el fuego?

En una grilla de $16 \times 16$ con el fuego en el centro medí cuántos pasos tarda en quemar el mapa según `pixel_scale` (los pies que representa cada celda):

**Cuadro 2:** Duración del incendio sin intervención según `pixel_scale` (grilla de $16 \times 16$, humedad 0.08).

| `pixel_scale` | Pasos hasta apagarse | Celdas quemadas |
|---|---|---|
| 50 | 19 | $\sim$252 (casi todo el mapa) |
| 150 | 29 a 34 | 253 a 256 |
| 300 | 64 a 79 | 256 (todo el mapa) |
| 600 | 12 a 13 | 64 a 80 (se apaga solo) |

Con `pixel_scale` bajo el fuego cruza el mapa en unos 20 pasos, demasiado rápido para un agente que coloca una celda por turno. Con 600 el fuego se apaga solo y el problema deja de tener sentido. Elegí 300: el fuego tarda entre 64 y 79 pasos en quemar todo el mapa si nadie interviene, lo que le da al agente margen para actuar.

### 4.3. Otras observaciones

Crear una simulación nueva tarda unos 0.03 segundos, así que puedo crear una por episodio, y un episodio completo dura aproximadamente 0.3 segundos. Además, según los logs el primer paso del fuego quema alrededor de 35 celdas sin que el agente pueda hacer nada, así que supongo que existe un piso de $\sim$35 celdas por episodio (esto es una deducción mía, no lo verifiqué directamente).

## 5. Baselines

Para saber si el agente aprende algo necesitaba referencias. Armé tres:

- **Sin acción:** no hago nada y el fuego se propaga libremente.

- **Azar:** cortafuegos en celdas libres elegidas al azar.
- **Heurística "perímetro":** en cada turno elijo al azar una celda libre que esté pegada (en las 8 direcciones) a una celda que arde. Si no hay ninguna, elijo una celda libre cualquiera. Es un baseline propio, no una técnica de la literatura: no usa el viento ni el combustible y no planifica.

Sobre 50 escenarios fijos dieron 212, 142 y 66 celdas quemadas respectivamente. O sea que el problema tiene margen (de 212 a 66) y que una regla muy simple ya hace casi todo el trabajo.

## 6. Algoritmos y arquitectura

Implementé DQN y Double DQN en PyTorch. Los dos usan replay buffer (30000 transiciones, guardadas en `float16` para ahorrar memoria), una red target que se copia cada 1000 pasos, política $\varepsilon$-greedy y pérdida de Huber con recorte de gradientes. El aprendizaje empieza después de 1000 pasos y hago una actualización cada 2 pasos del entorno. La única diferencia entre los dos algoritmos es el cálculo del target:

$$\text{DQN: } y = r + \gamma \max_a Q_{\text{target}}(s', a) \qquad \text{Double DQN: } y = r + \gamma\, Q_{\text{target}}\big(s', \arg\max_a Q_{\text{online}}(s', a)\big)$$

En los dos casos el máximo se calcula solo sobre las acciones válidas.

**Red.** Es totalmente convolucional con arquitectura *dueling*: cuatro capas convolucionales (una dilatada), una convolución $1 \times 1$ que da la ventaja $A(s,a)$ de cada celda y una rama que estima el valor del estado $V(s)$. La salida es $Q(s,a) = V(s) + A(s,a) - \overline{A}(s)$. Elegí una red totalmente convolucional (y no una CNN con capa densa final, estilo Atari) porque comparte pesos entre todas las celdas: "poner un cortafuego al lado del fuego" vale lo mismo en cualquier parte del mapa.

```
def forward(self, x):
    x = F.relu(self.conv1(x))
    x = F.relu(self.conv2(x))
    x = F.relu(self.conv3(x))
    if TIPO_RED == "fc":
        x = F.relu(self.fc1(x.flatten(1)))
        return self.fc2(x)
    x = F.relu(self.conv4(x))
    # Arquitectura dueling: Q(s,a) = V(s) + A(s,a) - promedio de A.
    # Separa "qué tan buena es la situación" (varía mucho entre estados) de "qué
    # celda conviene" (la diferencia chica entre acciones, que es lo que importa).
    ventaja = self.salida(x).flatten(1)
    valor = self.valor(x.mean(dim=(2, 3)))
    return valor + ventaja - ventaja.mean(dim=1, keepdim=True)
```

**Hiperparámetros finales:** learning rate $5 \cdot 10^{-5}$ (Adam), $\gamma = 0{,}9$, batch 64, $\varepsilon$ de 1.0 a 0.05 en 40000 pasos, 1500 episodios, y 3 semillas por agente.

```
def aprender(self):
    s, a, r, s2, terminado = self.buffer.muestrear(BATCH)
    q_actual = self.red(s).gather(1, a.unsqueeze(1)).squeeze(1)

    with torch.no_grad():
        validas_s2 = s2[:, 0].flatten(1) > 0.5
        q_target = self.red_target(s2)
        if self.usar_double:
            # Double DQN: la red online elige la acción, la target la evalúa
            q_online = self.red(s2).masked_fill(~validas_s2, -1e9)
            mejor_a = q_online.argmax(1, keepdim=True)
            q_siguiente = q_target.gather(1, mejor_a).squeeze(1)
        else:
            # DQN clásico: máximo de la red target
            q_siguiente = q_target.masked_fill(~validas_s2, -1e9).max(1)[0]
        q_siguiente = torch.where(validas_s2.any(1), q_siguiente, torch.zeros_like(q_siguiente))
```

```
        objetivo = r + self.gamma * q_siguiente * (1 - terminado)

    perdida = F.smooth_l1_loss(q_actual, objetivo)
    self.optimizador.zero_grad()
    perdida.backward()
    torch.nn.utils.clip_grad_norm_(self.red.parameters(), 10.0)
    self.optimizador.step()
    return perdida.item()
```

# 7. Historial de pruebas y correcciones

Esta fue la parte más larga del trabajo. El agente no aprendió a la primera, y fui probando y corrigiendo. Para saber si aprendía, no me alcanzaba con mirar la curva de entrenamiento: está mezclada con las acciones al azar de $\varepsilon$-greedy. Por eso agregué una **evaluación periódica sin azar** sobre escenarios fijos (cada 100 episodios) y la comparaba contra los baselines (azar $\sim$142, heurística $\sim$66). Después le sumé una segunda métrica de diagnóstico: el **porcentaje de cortafuegos colocados pegados al fuego**, que me dice directamente si el agente aprendió el comportamiento que premia el bonus.

**Cuadro 3:** Resumen de las pruebas. "EVAL" es el área quemada media sin azar sobre escenarios fijos.

| # | Qué probé | Qué pasó | Qué hice |
|---|---|---|---|
| 1 | CNN con capa densa, learning rate $10^{-4}$, $\gamma = 0{,}99$, solo recompensa de área (prueba local) | No aprendía: el área de entrenamiento seguía en $\sim$150 aunque $\varepsilon$ ya era bajo (a $\sim$750 episodios) | Pasar a una red totalmente convolucional |
| 2 | Red convolucional, mismos valores, DQN y Double DQN en Colab | EVAL entre 119 y 171 sin tendencia, igual que el azar | Agregar un bonus por cortafuego pegado al fuego (0.1) |
| 3 | + bonus de 0.1 | EVAL entre 138 y 158 hasta el episodio 700: seguía sin aprender | Cortar la corrida; cambiar varias cosas juntas (siguiente fila) |
| 4 | + red dueling, $\gamma = 0{,}9$, bonus 0.3, métrica de diagnóstico | Aprendió rápido (EVAL 107 en el ep. 100 y 67 en el 200) pero después se degradó a $\sim$100, y los cortafuegos pegados al fuego bajaron de 65 % a $\sim$40 % | Atacar la inestabilidad |
| 5 | Learning rate $5 \cdot 10^{-5}$ y target cada 1000 pasos | Estable: EVAL entre 59 y 79 durante los 1500 episodios | Dejar estos valores |
| 6 | Selección del mejor checkpoint con escenarios de validación separados, 3 semillas y 4 agentes | Resultados finales (secciones 8 y 9) | — |

## 7.1. Reward shaping

Con la recompensa original (solo área quemada) el agente aprendía muy poco. Mi hipótesis, que no probé de forma aislada, es que el efecto de un cortafuego aparece varios pasos después y queda tapado por el ruido de todas las demás celdas que se queman en cada paso. Por eso agregué un bonus inmediato de $+0{,}3$ cuando el cortafuego queda pegado a una celda que arde. Es una recompensa extra que solo se usa para entrenar: las curvas y las evaluaciones miden siempre el área quemada, sin el bonus, así que la comparación contra los baselines es justa.

```
def step(self, accion):
    """accion = índice de celda (0..TAM*TAM-1) o None para no poner nada."""
    recompensa = 0.0   # parte "real" de la recompensa (área quemada)
    bonus = 0.0        # parte del reward shaping
    if accion is not None:
```

```
        fila, columna = divmod(int(accion), TAM)
        if self.sim.fire_map[fila, columna] == SIN_QUEMAR:
            # ¿hay alguna celda ardiendo pegada a esta (8 vecinas)?
            vecinas = self.sim.fire_map[max(0, fila - 1):fila + 2, max(0, columna - 1):columna + 2]
            pegada_al_fuego = bool((vecinas == QUEMANDOSE).any())
            if pegada_al_fuego:
                bonus = self.bonus
                self.total_frontera += 1
            # update_mitigation espera (columna, fila, tipo)
            self.sim.update_mitigation([(columna, fila, BurnStatus.FIRELINE)])
            self.lineas += 1
            self.total_lineas += 1
        else:
            recompensa -= 0.5   # celda inválida (el agente la evita con la máscara)

    _, self.activo = self.sim.run(1)    # el fuego avanza 1 minuto
    self.pasos += 1

    terminado = (not self.activo) or (self.pasos >= MAX_PASOS)
    if terminado and self.activo:
        # Se acabaron los turnos pero el fuego sigue: lo dejo terminar para poder
        # medir el área final (sin poner más cortafuegos)
        extra = 0
        while self.activo and extra < 500:
            _, self.activo = self.sim.run(1)
            extra += 1

    quemadas_ahora = self.area_quemada()
    recompensa -= 0.1 * (quemadas_ahora - self.quemadas_antes)
    self.quemadas_antes = quemadas_ahora

    if not (self.sim.fire_map == SIN_QUEMAR).any():
        terminado = True
    self.ret_area += recompensa
    return self.observar(), recompensa + bonus, terminado
```

## 7.2. Inestabilidad y selección de checkpoint

Cuando por fin aprendió (prueba 4), el desempeño se degradaba con el entrenamiento: el agente era muy bueno a los 100–200 episodios y después empeoraba. Bajé el learning rate y dejé más quieta la red target (prueba 5), lo que lo estabilizó. Además decidí **guardar los pesos del mejor momento** según un conjunto de validación de 40 escenarios, distinto del de la evaluación final, y usar esos pesos al final.

```
if (ep + 1) % EVAL_CADA == 0:
    estado_azar = np.random.get_state()   # guardo el azar para no alterar el entrenamiento
    entorno_eval.total_lineas = 0
    entorno_eval.total_frontera = 0
    _, areas_eval = evaluar(entorno_eval, lambda e, o: agente.elegir_accion(o, 0.0),
                            SEMILLAS_VALIDACION)
    np.random.set_state(estado_azar)
    agente.evals.append((ep + 1, float(areas_eval.mean())))
    if areas_eval.mean() < agente.mejor_area:
        agente.mejor_area = float(areas_eval.mean())
        agente.mejor_ep = ep + 1
        agente.mejor_pesos = copy.deepcopy(agente.red.state_dict())
```

Una consecuencia que noté después: casi todos los mejores checkpoints caen entre los episodios 100 y 400, o sea que la mayor parte del aprendizaje ocurre al principio.

## 7.3. Un experimento más: bonus decreciente

Como el agente con shaping solo igualaba a la heurística, probé si podía mejorarla haciendo que el bonus baje con el tiempo (de 0.3 en el episodio 300 a 0.05 en el 800), para que al final mande la

recompensa de área. La idea era que primero aprendiera “cortafuego al lado del fuego” y después refinara la política. No funcionó, como cuento en los resultados.

```
if bonus_fin is not None:
    # bonus decreciente: constante hasta DECAE_DESDE y después baja linealmente
    # hasta bonus_fin en DECAE_HASTA. La idea es que primero aprenda "poner el
    # cortafuego al lado del fuego" y después la recompensa de área refine la política.
    fraccion = min(1.0, max(0.0, (ep - DECAE_DESDE) / (DECAE_HASTA - DECAE_DESDE)))
    entorno.bonus = bonus + fraccion * (bonus_fin - bonus)
```

# 8. Protocolo de evaluación

- **Entrenamiento:** 1500 episodios con escenarios aleatorios, 3 semillas por agente (0, 1 y 2).
- **Validación:** cada 100 episodios, 40 escenarios fijos, sin azar. Sirve para elegir el checkpoint.
- **Evaluación final:** 50 escenarios fijos que no se usaron para entrenar ni para elegir el checkpoint. Todos los agentes y baselines se evalúan sobre los mismos escenarios. Los resultados juntan las 3 semillas de cada agente.
- **Agentes comparados:** DQN y Double DQN con shaping; DQN sin shaping; y DQN con bonus decreciente.

# 9. Resultados

**Cuadro 4:** Evaluación final en 50 escenarios no vistos (media ± desvío entre escenarios). Área quemada sobre un máximo de 256 celdas.

| Política | Recompensa | Área quemada |
|---|---|---|
| Sin acción | $-21{,}10 \pm 8{,}20$ | $212{,}0 \pm 82{,}0$ |
| Azar | $-14{,}14 \pm 5{,}43$ | $142{,}4 \pm 54{,}3$ |
| Heurística perímetro | $-6{,}51 \pm 3{,}75$ | $66{,}1 \pm 37{,}5$ |
| DQN (con shaping) | $-7{,}52 \pm 4{,}74$ | $76{,}2 \pm 47{,}4$ |
| Double DQN (con shaping) | $-7{,}39 \pm 4{,}45$ | $74{,}9 \pm 44{,}5$ |
| DQN sin shaping | $-12{,}17 \pm 5{,}13$ | $122{,}7 \pm 51{,}3$ |
| DQN bonus decreciente | $-7{,}32 \pm 4{,}43$ | $74{,}2 \pm 44{,}3$ |

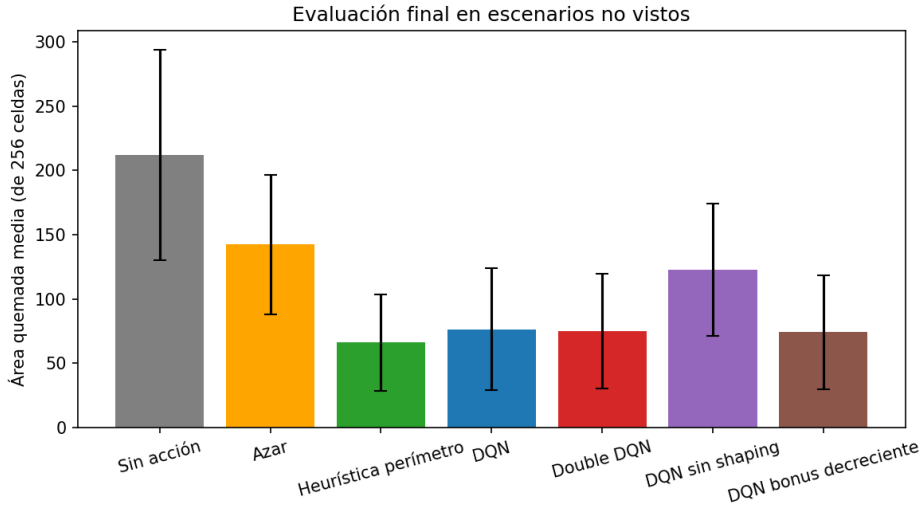



**Figura 1:** Evaluación final. Las barras de error son el desvío entre escenarios, no el error de la media.

## 9.1. Curvas de convergencia

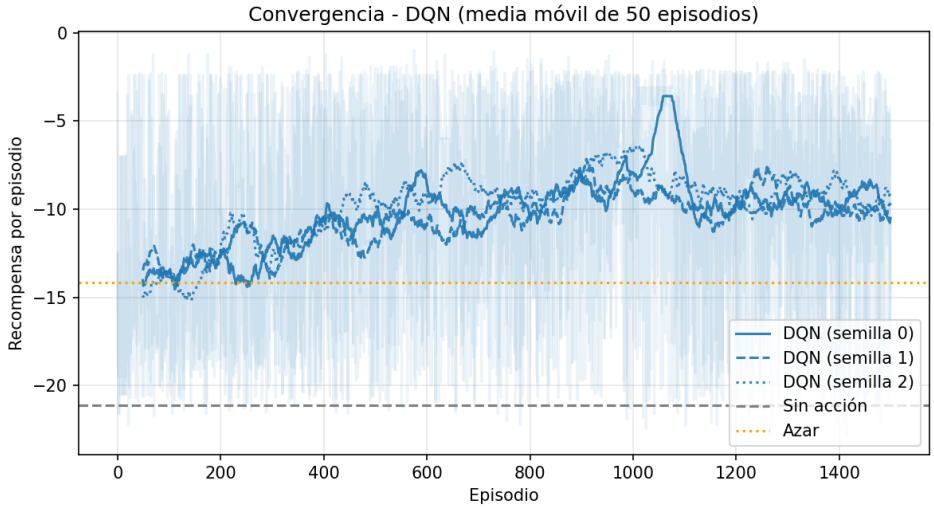



**Figura 2:** Convergencia de DQN: recompensa por episodio (media móvil de 50), una línea por semilla. La curva cruda, muy ruidosa, se ve de fondo porque cada escenario es distinto: algunos incendios se apagan casi solos y otros arrasan el mapa.

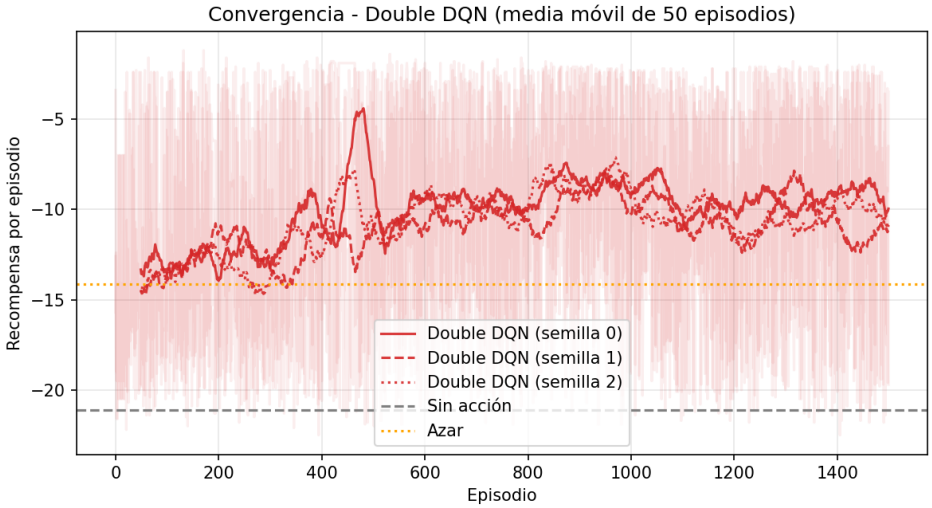



**Figura 3:** Convergencia de Double DQN, mismo formato.

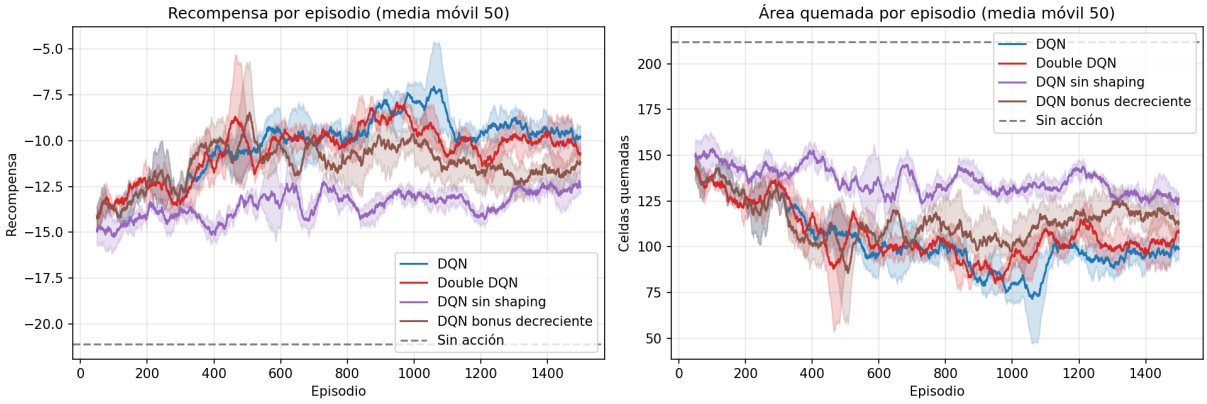



**Figura 4:** Comparativa de los cuatro agentes: recompensa y área quemada por episodio (media móvil de 50; la línea es la media de las 3 semillas y la banda el desvío entre semillas).

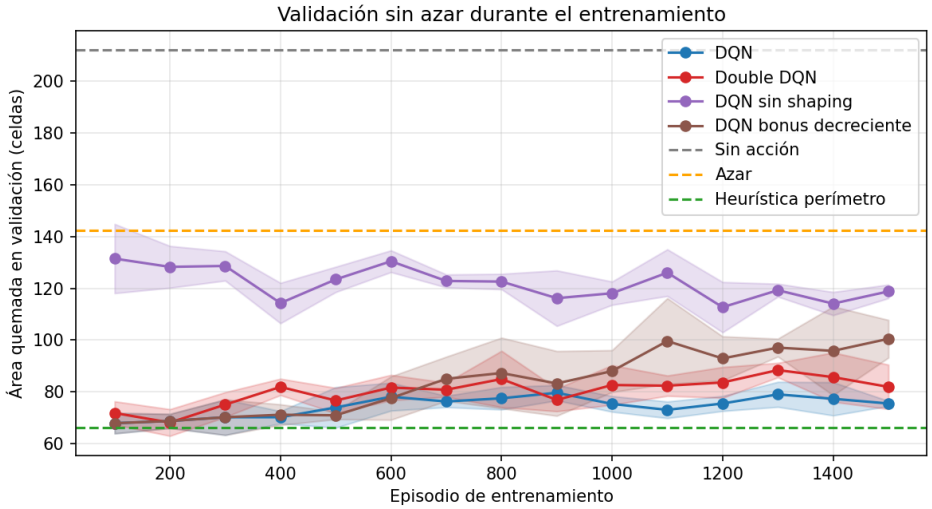



**Figura 5:** Área quemada en validación, sin azar, durante el entrenamiento, con las referencias de los baselines. Esta es la curva que mejor muestra si el agente aprendió.

### 9.2. Mejor checkpoint contra último episodio

Para cada agente y semilla, la validación (40 escenarios) del mejor checkpoint y la del último episodio:

**Cuadro 5:** Validación del mejor checkpoint (con el episodio entre paréntesis) y del episodio 1500, por semilla (0, 1, 2).

| Agente | Mejor checkpoint | Episodio 1500 |
|---|---|---|
| DQN | 69.0 (300), 65.0 (200), 62.5 (300) | 74.2, 75.7, 76.7 |
| Double DQN | 62.2 (200), 67.4 (200), 69.2 (100) | 69.8, 88.7, 87.2 |
| DQN sin shaping | 105.0 (400), 99.1 (1200), 103.0 (900) | 120.2, 115.1, 121.0 |
| DQN bonus decreciente | 65.5 (400), 65.0 (200), 62.5 (300) | 94.2, 110.7, 96.5 |

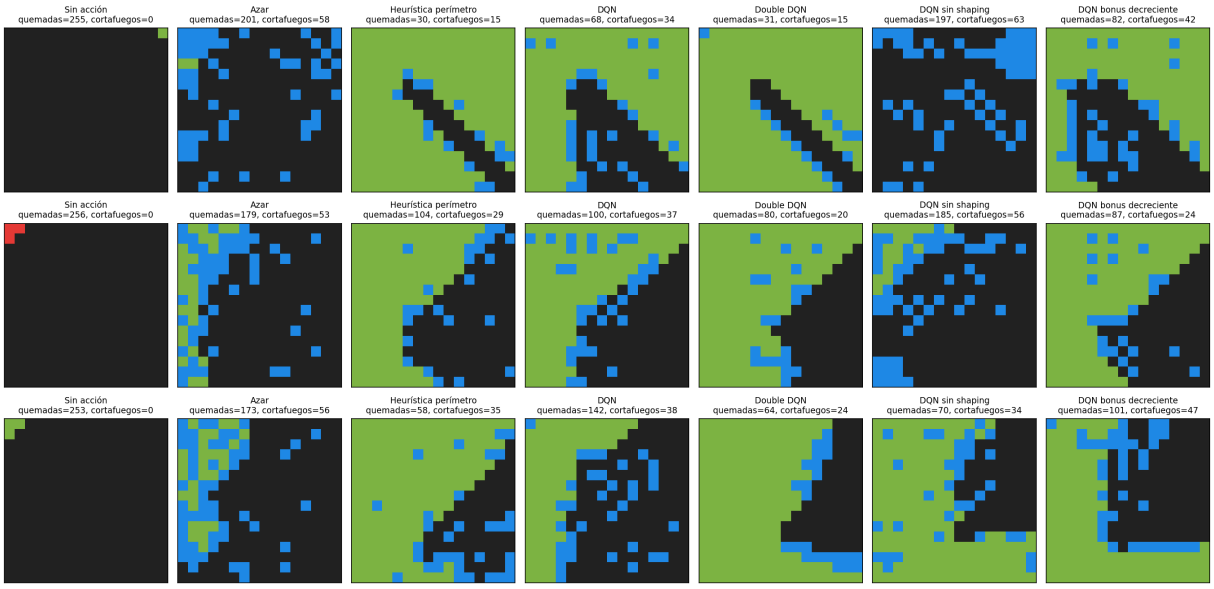



**Figura 6:** Mapa final de tres escenarios con cada política (los agentes son los de la semilla 0). Verde: sin quemar; negro: quemado; azul: cortafuego; rojo: ardiendo. Sobre cada mapa se indican las celdas quemadas y los cortafuegos usados.

## 10. Análisis y discusión

- **El agente aprendió, y mucho.** Con shaping, DQN (76.2) y Double DQN (74.9) bajan el área quemada de 142 (azar) a ∼75 y de 212 (sin hacer nada). El resultado es consistente en las tres semillas.
- **No supera a la heurística.** Quedan a ∼10 celdas de la heurística (66.1). Con un desvío entre escenarios de ∼45, esa diferencia no es significativa, pero tampoco puedo afirmar que la igualen. Es esperable que la igualen y no la superen: el bonus premia justamente el criterio de la heurística (cortafuego pegado al fuego).
- **DQN contra Double DQN.** La diferencia promedio es de 1.3 celdas (76.2 contra 74.9), muy chica frente al desvío de ∼45, así que **no puedo decir que Double DQN sea mejor**. En los tres escenarios de la Figura 6 Double DQN (semilla 0) quemó menos que DQN (semilla 0) (31 contra 68, 80 contra 100 y 64 contra 142), pero son solo tres escenarios y una semilla. En la validación, además, Double DQN termina algo más alto que DQN en

las semillas 1 y 2 (88.7 y 87.2 contra 75.7 y 76.7), y más bajo en la semilla 0 (69.8 contra 74.2). Mi conclusión es que son equivalentes en este problema.

- **El shaping hace la diferencia.** Sin shaping el agente mejora al azar pero queda en ∼123, y apenas el 13 a 31 % de sus cortafuegos van pegados al fuego. Ojo: con los hiperparámetros finales el agente sin shaping sí mejora lentamente, mientras que con los primeros no aprendía nada (prueba 2), ni siquiera con un bonus chico de 0.1 (prueba 3). Como cambié varias cosas a la vez en la prueba 4 (dueling, $\gamma$ y bonus), no sé cuál de ellas fue la clave del salto.
- **Bonus decreciente (resultado negativo).** En la tabla final da 74.2, casi igual a DQN, pero eso engaña: el checkpoint elegido de cada semilla (episodios 400, 200 y 300) es anterior o coincide con el inicio de la bajada del bonus. En la validación (Figura 5) el agente empeora hasta ∼100 celdas en el episodio 1500 (contra ∼75 de DQN), y el porcentaje de cortafuegos pegados al fuego cae de ∼75 % a ∼40 %. El agente depende del bonus: cuando se lo saco, se olvida lo aprendido, y la recompensa de área no alcanza para sostenerlo. No se puede afirmar que mejore a DQN.
- **Qué hace el agente.** En los mapas (Figura 6, primer escenario) la heurística y Double DQN cierran el fuego con una línea diagonal (30 y 31 celdas quemadas), mientras que el azar y el agente sin shaping dispersan cortafuegos que no contienen nada (201 y 197). Es un escenario ilustrativo, no un promedio.
- **La curva de entrenamiento engaña un poco.** La recompensa mejora de $-14$ a $-10$ en parte porque $\varepsilon$ baja y hay menos acciones al azar. La validación sin azar muestra que casi todo se aprende en los primeros 100 a 300 episodios y después queda plano.

## 11. Limitaciones

- Grilla chica ($16 \times 16$), terreno fijo y una sola acción (cortafuego de un tipo). No sé cómo se comportaría en mapas más grandes o con terreno variable.
- Solo 3 semillas por agente y 50 escenarios de evaluación. Los desvíos son entre escenarios (no el error de la media) y no hice pruebas estadísticas formales, por eso evito hablar de diferencias significativas.
- El shaping hace que el agente tienda a imitar la heurística, así que no demostré que aprenda algo más sofisticado que ella.
- La selección del mejor checkpoint se hace sobre un conjunto de validación, lo que favorece un poco a los mejores valores; la evaluación final usa otros escenarios para evitar eso.
- Los experimentos previos del simulador (sección 4) y la prueba de la CNN con capa densa se hicieron localmente y no están en el repositorio; la comparación de la capa densa no se repitió en Colab.
- Mi hipótesis de por qué sin shaping se aprende poco (el efecto del cortafuego es tardío y ruidoso) no la probé de forma aislada.

## 12. Conclusiones y trabajo futuro

Logré entrenar agentes que contienen incendios simulados bastante mejor que el azar o que no intervenir: de 212 celdas quemadas sin hacer nada y 142 con el azar, a ∼75 con DQN y Double DQN con reward shaping, cerca de una heurística simple de 66. Lo que más aprendí en el camino:

- **Calibrar el entorno antes de entrenar** me ahorró mucho tiempo: sin saber que con humedad 0.03 los cortafuegos se quemaban, o que con `pixel_scale` bajo el fuego era demasiado rápido, habría culpado al algoritmo por problemas del simulador.
- **Hay que mirar la métrica correcta.** La curva de entrenamiento me daba una falsa idea; la evaluación sin azar y el porcentaje de cortafuegos pegados al fuego me mostraron lo que pasaba de verdad.
- **Una recompensa esparsa y tardía puede impedir el aprendizaje** aunque el algoritmo esté bien implementado, y un bonus bien elegido lo destraba, aunque a cambio el agente depende de él (el experimento del bonus decreciente lo mostró).
- **DQN y Double DQN rindieron igual** en este problema; no encontré motivos para preferir uno.

Como trabajo futuro probaría retornos de $n$ pasos y replay priorizado para mejorar la asignación de crédito sin depender del bonus, terreno y combustible variables, mapas más grandes, y los otros tipos de mitigación de SimFire (línea mojada y línea de rascado), además de más semillas y una comparación estadística formal entre agentes.

## 13. Cómo reproducirlo

El notebook `incendios_dqn_TP_FINAL.ipynb` del repositorio corre en Google Colab y se instala solo. Conviene activar GPU (T4). La celda de parámetros permite:

- `MODO_RAPIDO = True` para una prueba corta de unos 2 minutos (sirve para confirmar que la instalación funciona);
- `SEMILLAS_ENTRENAMIENTO` para elegir cuántas semillas por agente;
- `SOLO_AGENTES` para entrenar solo algunos agentes;
- `GUARDAR_EN_DRIVE = True` para no perder los resultados si se cae la sesión.

Según los logs, cada entrenamiento de 1500 episodios tarda entre 6 y 10 minutos; la corrida completa de 4 agentes con 3 semillas llevó alrededor de 1.5 a 2 horas. Al terminar, el notebook genera los gráficos, un `resumen.json`, las curvas de entrenamiento y los pesos de cada agente.